and cleaning up code apply to tests. This is especially true of the general coding guideline that a method should only do one thing, which here means splitting up test setups into semantically meaningful parts, as well as keeping each test focused on one particular goal.

On the other hand, if you can't write short tests, consider the possibility that it is the code's fault and you need to do some redesign. If it's hard to set up a short test, that often indicates the code has too many internal dependencies.

Brian W. Kernighan and P.J. Plauger state in *The Elements of Programming Style [KP78]*, "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it." Because tests don't have their own tests, this quote suggests that you should keep your tests simple to give yourself cognitive room to understand them.

In particular, this guideline argues against using clever tricks to reduce duplication among multiple tests that share a similar structure. If you find yourself starting to metaprogram to generate multiple tests in your suite, you'll probably find that complexity working against you at some point. You never want to have to decide whether a bug is in your test or in the code. And when—not if—you do find a bug in your test suite, it's easier to fix if the test code is simple.

At some point in your testing career, you'll have a series of key/value pairs to test, and you'll be tempted to do something like this (this example is deliberately simplified):

```
DATA = {admin: 5, super_admin: 20, user: 3}

DATA.each_pair do |key, value|
  it "correctly determines the threshold for #{key} users" do
    user = User.new(role: key)
    expect(user.threshold).to eq(value)
  end
end
```

This will work; RSpec is flexible enough to programmatically create it blocks inside a loop. And while this is "good code" according to normal code metrics—it removes duplication while maintaining a single assertion—it's also a little opaque, especially if the body of the loop or the data gets more complicated. When this code fails, it's a little harder to see what happens. Specifically, RSpec will give you a failure message with a line number and it will be hard to tell from that information which iteration of the loop has failed, and hard to run that iteration without running all of the others.